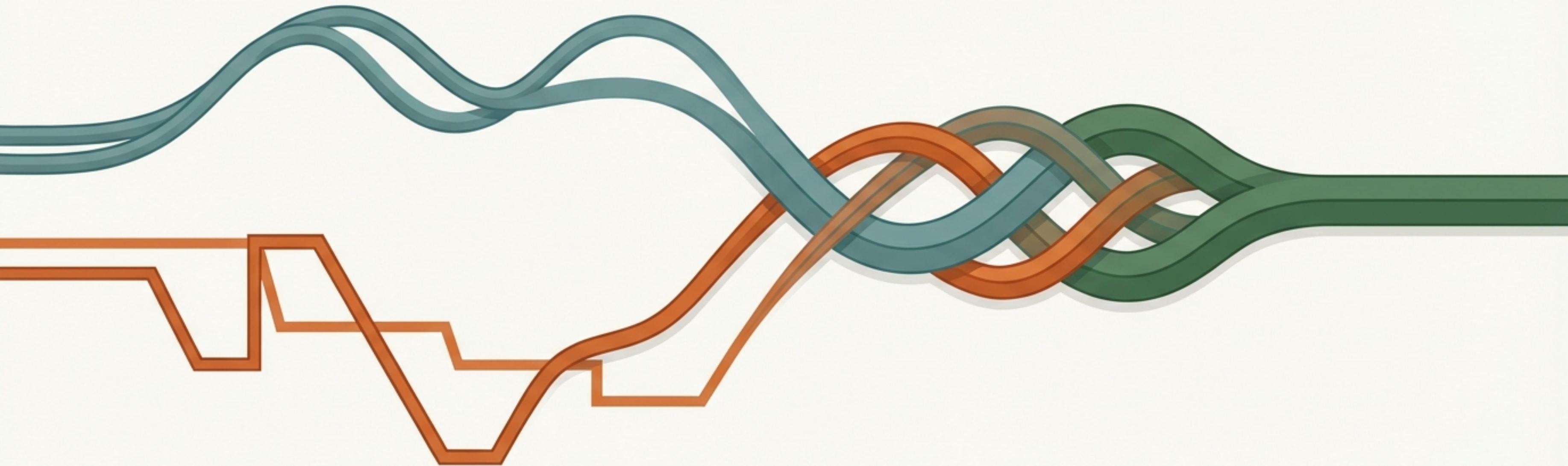
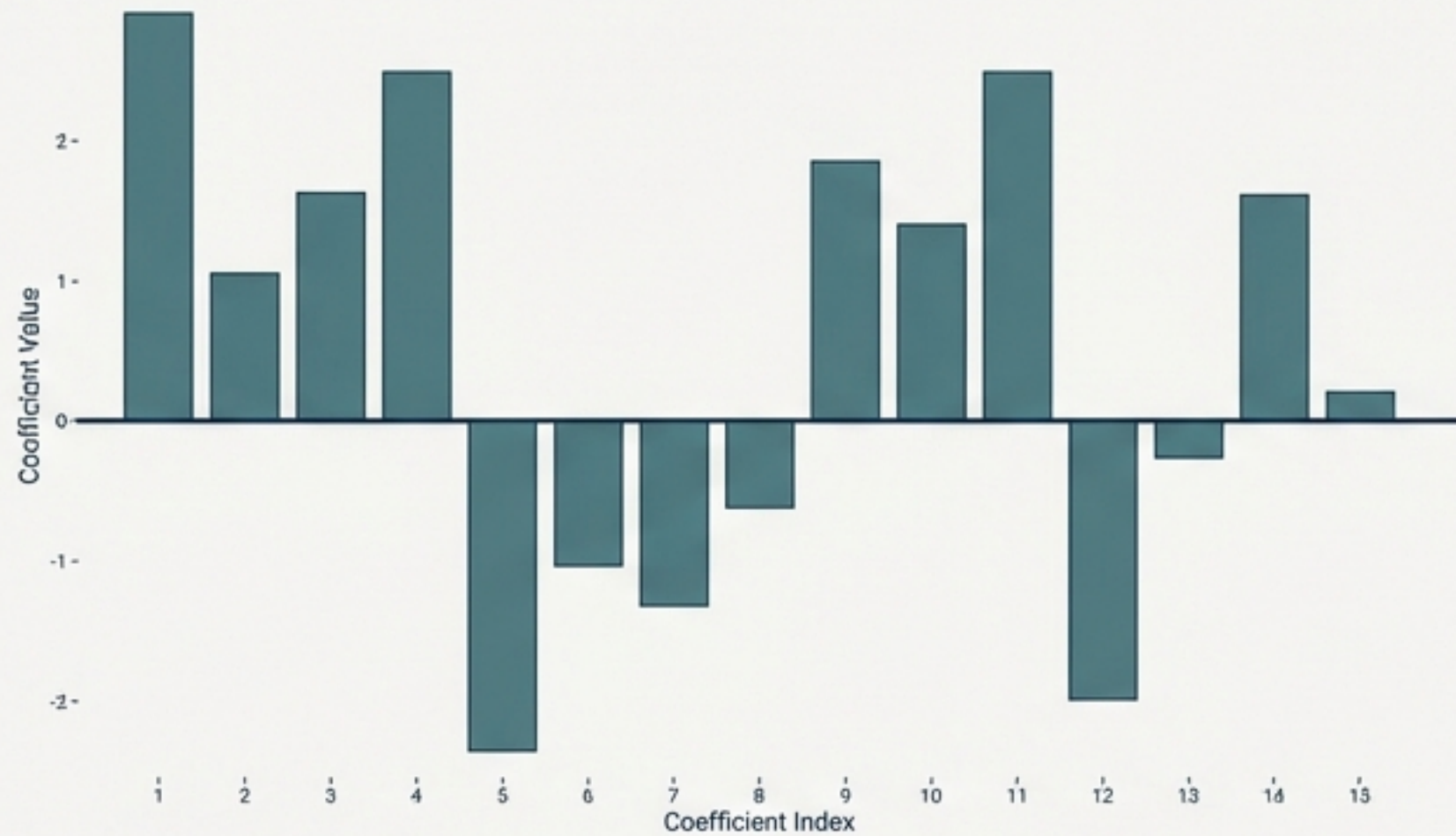


Mastering ElasticNet Regression

A visual guide to combining Ridge and Lasso for high-dimensional data

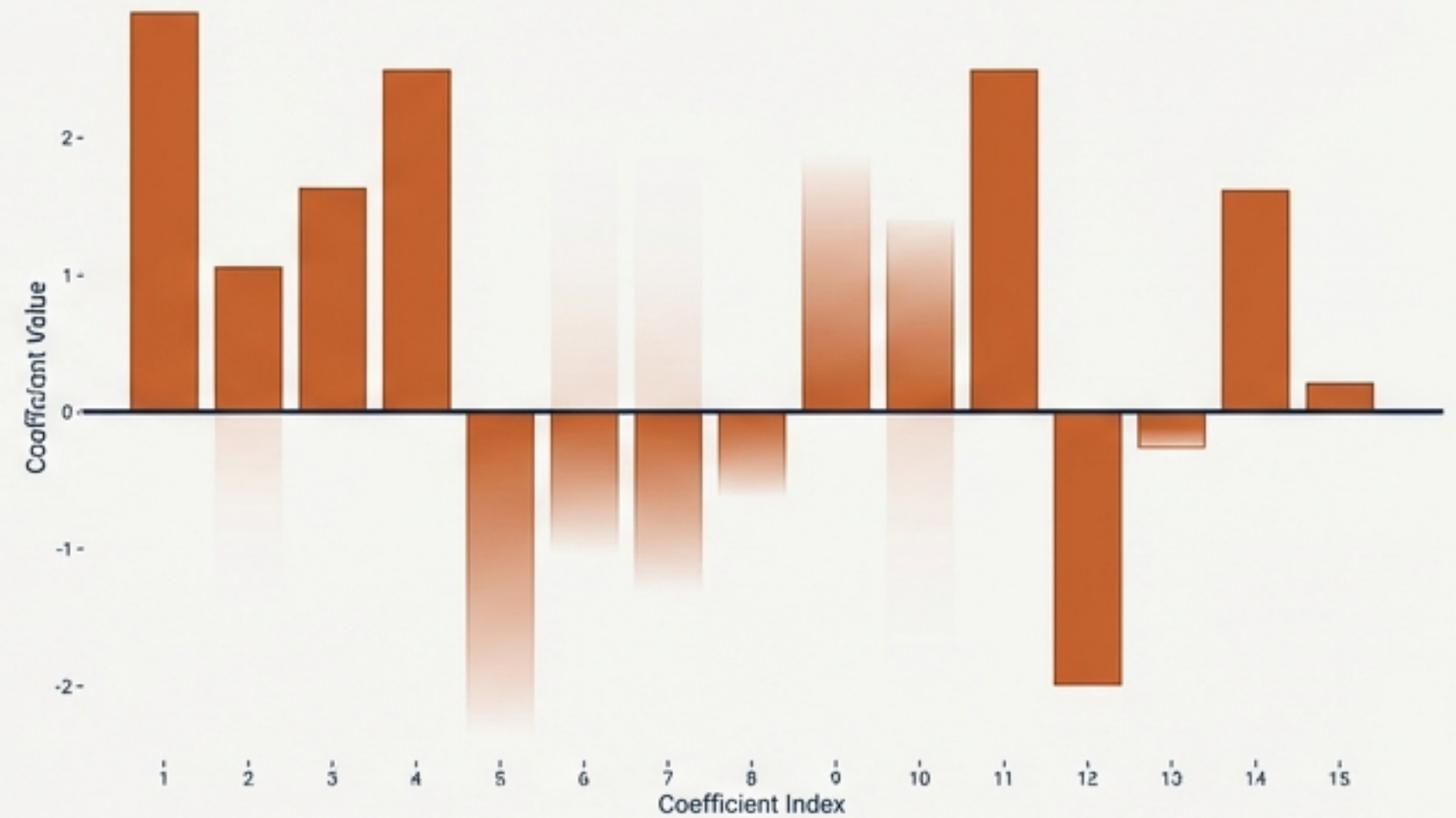


The foundational regularisation techniques



Ridge (L2)

Shrinks all coefficients to reduce overfitting.
Best used when all input columns are important to the model.



Lasso (L1)

Drives unimportant coefficients to absolute zero. Acts as built-in feature selection when only a subset of columns is important.

The high-dimensional blind spot

What happens when you have a massive dataset with $N \gg 50$ columns?

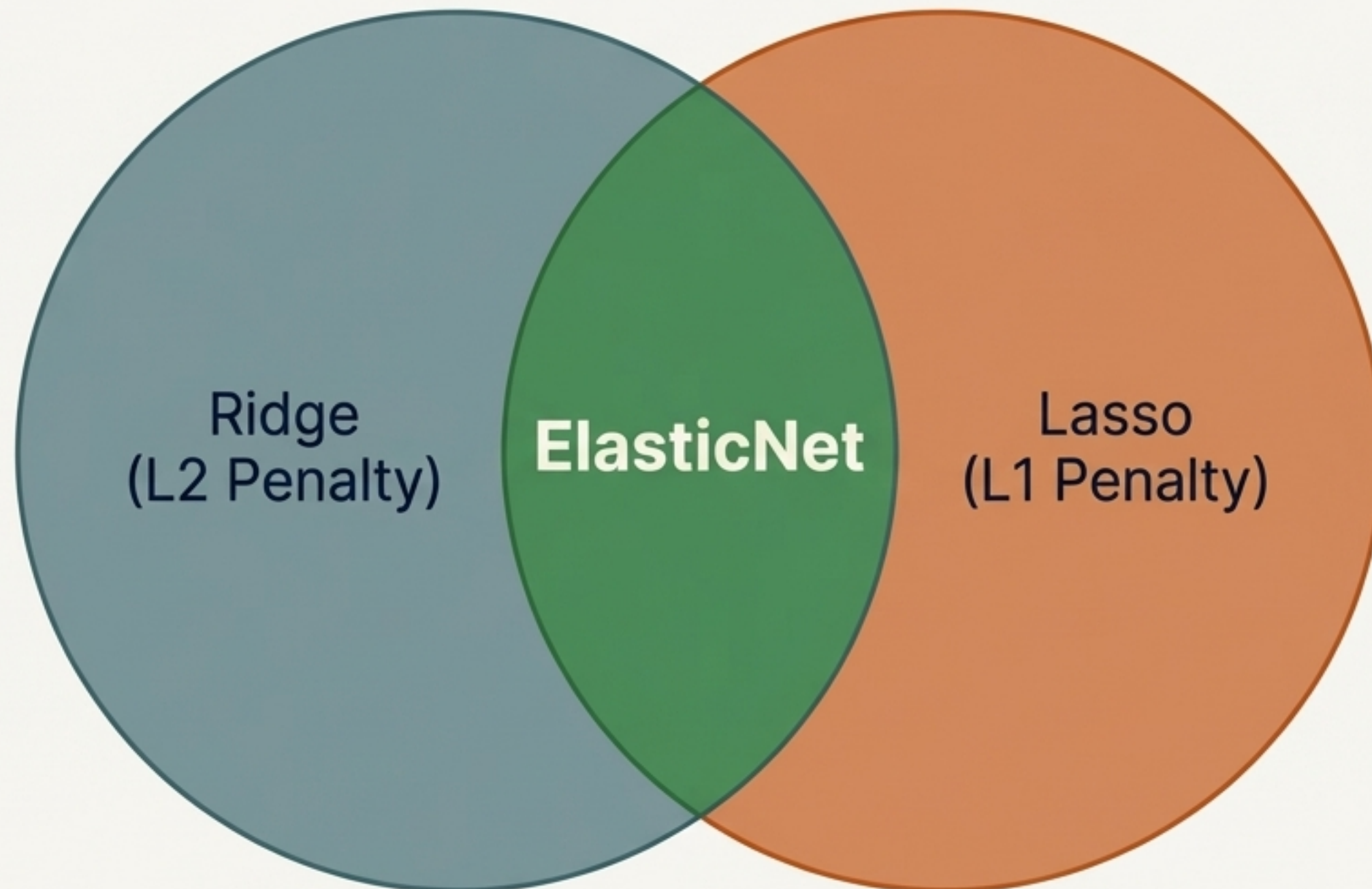
You cannot know in advance the exact importance of every single feature. Choosing purely **Ridge** risks keeping useless noise. Choosing purely Lasso risks blindly discarding valuable nuanced data.



ElasticNet: The elegant synthesis

ElasticNet does not force you to choose.

It is a hybrid regularisation technique that applies both penalties simultaneously. It retains the stability of Ridge while maintaining the feature selection capability of Lasso.



The mathematics of synthesis

The loss function simply adds both regularisation terms to the base error. The critical additions are parameters a and b , which determine exactly how much of each penalty is applied.

Mean Squared Error (MSE)

$$\sum (y_i - \hat{y}_i)^2 + a \times \left(\text{Ridge Penalty} \right) + b \times \left(\text{Lasso Penalty} \right)$$

The structural
hyperparameters

Translating theory into Scikit-learn parameters

Scikit-learn uses simple algebra to extract two unknowns from two hyperparameters. If you know your desired overall penalty and your desired balance, the model perfectly allocates the Ridge and Lasso weights.

$$a \times (\text{Ridge Penalty}) + b \times (\text{Lasso Penalty})$$

$$\text{alpha} = a + b$$

Determines the total regularisation strength applied to the model.

$$\text{l1_ratio} = \frac{a}{a + b}$$

Determines the balance and weighting between the two penalties.

The l1_ratio mixing desk

Think of l1_ratio as a dial. By default, it applies an **equal 50/50 split**. Adjusting the dial allows you to assign greater weight to Ridge or Lasso depending on your dataset's behaviour.

Set to 0.9: Heavily favouring
feature selection while
retaining slight Ridge stability.



0.0 (Pure Ridge)

0.5 (Equal Split)

1.0 (Pure Lasso)

Taming multicollinearity

- **The Problem:** When input columns are highly dependent on one another.
- **The Solution:** ElasticNet mathematically outperforms pure Ridge or pure Lasso by grouping correlated features together rather than randomly dropping one.

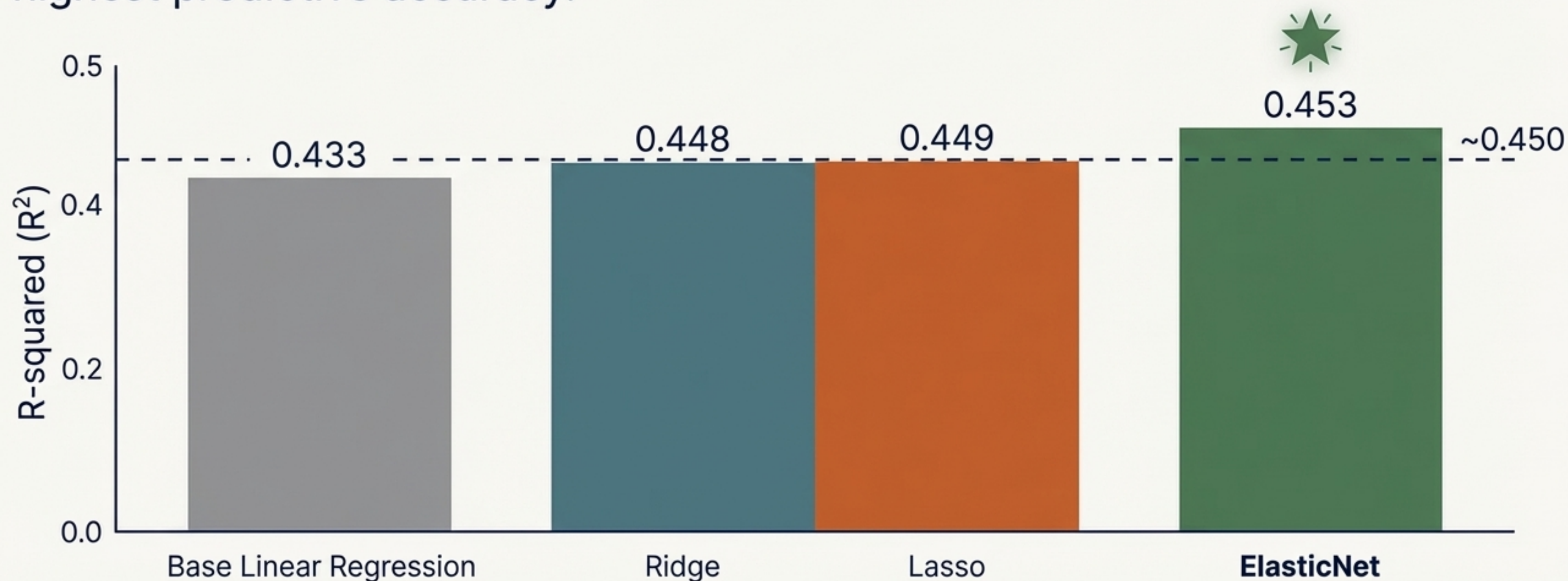


The regularisation selection matrix

Model	Use When	Core Benefit
Ridge (L2)	All features are important	Prevents overfitting
Lasso (L1)	Only a subset of features is important	Automatic feature selection
ElasticNet	Massive dataset (importance unknown) OR high multicollinearity	Best of both worlds

Real-world performance gains

Testing on the Diabetes dataset proves the theory. By fine-tuning the hyperparameters ($\alpha=0.005$, $l1_ratio=0.9$), ElasticNet extracts the highest predictive accuracy.



Two paths to implementation

Scikit-learn provides two ways to apply ElasticNet. While **SGDRegressor** allows penalty arguments, the dedicated **ElasticNet** class is highly optimised for this specific mathematical operation.

Path A (Preferred)



```
from sklearn.linear_model import ElasticNet  
  
model = ElasticNet(alpha=0.005, l1_ratio=0.9)
```

Path B

```
from sklearn.linear_model import SGDRegressor  
  
model = SGDRegressor(penalty='elasticnet')
```

'elasticnet'

The 50-column challenge

Put it to the test:

1. Find a dataset with 50+ columns.
2. Apply Ridge, Lasso, and ElasticNet separately.
3. Document the results.

Experience firsthand how ElasticNet handles the complexity of high-dimensional data better than standalone regularisation.

